CI/CD Pipeline & Test Automation

A plain-language explanation of the project, its components, the end-to-end workflow and the architecture

The project statement

> *"Raised test coverage to 85% and shipped daily instead of weekly by integrating CI/CD (GitHub Actions, Docker, Kubernetes) with Pytest integration tests and leading design and code reviews."*

Technology	Role in one phrase
GitHub Actions	The robot that runs every check automatically
Docker	Packs the app into an identical box that runs anywhere
Kubernetes	Runs those boxes in production and keeps them healthy
Pytest	Writes and runs the automated tests
Design & code reviews	Humans checking plans and code before they ship

Contents

1. Overall Summary (in simple terms) — The project in one paragraph, an analogy, results and assumptions

2. Key Terms in Plain English — A glossary of every technical word used

3. The Components, Explained One by One — Eleven components, each with minute details

4. The End-to-End Workflow, Step by Step — Seventeen steps from keyboard to users

5. Architecture Diagram — The diagram, every zone and every arrow explained

6. How the Two Results Were Achieved — 85% coverage and daily releases

7. Explaining It in an Interview — A 30-second pitch and likely questions

1. Overall Summary (in simple terms)

> **The project in one paragraph**
>
> Before this project, the team released new software **once a week**. Testing was partly manual, many parts of the code had no automated tests, and every release was a big, nervous event. I built an **automatic assembly line** for code. Now, every time a developer saves a change to GitHub, a robot (**GitHub Actions**) automatically checks the code style, runs hundreds of **Pytest** tests (including **integration tests** that check the parts work together with a real database), confirms that at least **85% of the code is tested**, packages the app into a **Docker** container, and rolls it out to **Kubernetes**, which runs it for users without any downtime. Alongside the automation, I led **design reviews** (agreeing on the plan before coding) and **code reviews** (a teammate checks every change). Because checks became fast, automatic and trustworthy, the team could safely ship **every day instead of every week**.

1.1 The restaurant analogy

Imagine the software team is a restaurant kitchen:

- **Developers** are the chefs. Each code change is a new dish.
- **GitHub** is the order board and recipe book. Every recipe (code) and every change to it is recorded there.
- **Design review** is the head chef approving the menu idea before anyone starts cooking.
- **Pytest tests** are the taste tests. **Unit tests** taste one ingredient; **integration tests** taste the full plate to check the ingredients work together.
- **Test coverage (85%)** is the share of the recipe that has actually been tasted. 85% means very little goes out untasted.
- **GitHub Actions** is a tireless kitchen inspector who tastes every dish automatically, every single time, without being asked.
- **Code review** is a second chef looking at the plate before it leaves the kitchen.
- **Docker** is a sealed lunchbox. The dish tastes exactly the same no matter which table it is served at.
- **Kubernetes** is the restaurant manager who decides how many waiters are needed, replaces a waiter who falls sick, and swaps old dishes for new ones without closing the restaurant.

Before: dishes were tasted by hand once a week, so the kitchen served new food weekly. After: every dish is tasted automatically within minutes, so new food can be served every day.

1.2 Results at a glance

Measure	Before	After	Why it changed
Test coverage	Low / patchy	85%+ (enforced)	New tests written for key paths; pipeline blocks changes that drop coverage
Release frequency	Weekly	Daily	Every change is tested, packaged and deployed automatically
Testing effort	Largely manual	Automated on every change	GitHub Actions runs Pytest on every push and pull request
Environment problems	“Works on my machine”	Same container everywhere	Docker packages the app with everything it needs
Deploy downtime	Possible during releases	Zero-downtime rollouts	Kubernetes rolling updates and automatic rollback
Code quality	Inconsistent	Reviewed and consistent	Design reviews, code reviews and automated linting

1.3 A note on assumptions

The project statement names GitHub Actions, Docker, Kubernetes and Pytest. A real pipeline also needs a few supporting pieces that the one-line statement does not name: a **linter** (style checker), a **coverage tool** (pytest-cov), a **container registry** (a store for Docker images, such as GitHub Container Registry, Docker Hub or AWS ECR) and **monitoring** (health checks and alerts). They are included here because the pipeline cannot work without them. Specific tool names for these supporting pieces are typical choices; you can swap in whichever ones your team actually used.

2. Key Terms in Plain English

Read this table first if any word in the document feels unfamiliar.

Term	Plain-English meaning
CI (Continuous Integration)	Every code change is automatically merged with everyone else's and tested right away, many times a day.
CD (Continuous Delivery / Deployment)	Every change that passes the tests is automatically packaged and released to users.
Pipeline	A fixed list of automatic steps that every code change goes through, like an assembly line.
Repository (repo)	The folder on GitHub that holds all the project's code and its full history.
Branch	A private copy of the code where a developer makes changes without disturbing anyone else.
main branch	The official version of the code. Whatever is here is what gets released.
Pull Request (PR)	A request saying "please review my changes and merge them into main".
Merge	Combining a branch's changes into main.
Unit test	A tiny automatic test that checks one small piece of code (one function) in isolation.
Integration test	A test that checks several pieces working together, e.g. the app plus a real database.
Test coverage	The percentage of code lines that are run at least once by the tests.
Coverage gate	A rule that fails the pipeline if coverage drops below a target (85% here).
Lint / linter	A tool that checks code style and spots simple mistakes without running the code.
Docker image	A frozen, ready-to-run package of the app plus everything it needs.
Container	A running copy of a Docker image.

Term	Plain-English meaning
Dockerfile	The recipe that tells Docker how to build the image.
Container registry	An online warehouse that stores Docker images by version.
Kubernetes (K8s)	A system that runs containers on many machines and keeps them healthy automatically.
Pod	The smallest unit Kubernetes runs; usually one container of the app.
Deployment (K8s)	The instruction “keep N copies of version X of my app running”.
Namespace	A separate area inside Kubernetes, e.g. one for staging and one for production.
Staging	A practice copy of production where changes are tried before real users see them.
Production	The live system that real users use.
Rolling update	Replacing old copies of the app with new ones a few at a time, so there is no downtime.
Rollback	Going back to the previous working version if the new one has problems.
Smoke test	A quick check right after deploying that the most basic things still work.
Runner	The temporary computer GitHub Actions uses to run the pipeline steps.
Secret	A password or key stored safely so the pipeline can use it without exposing it.

3. The Components, Explained One by One

Each component below follows the same pattern: what it is in one line, an everyday analogy, its job in this project, the minute details of how it works, and why it matters.

3.1. Application Code (Python)

In one line: The actual software the team builds, written in Python (for example a web service or API).

Everyday analogy: The dish the kitchen is cooking. Everything else in this project exists to make sure this dish is safe and served quickly.

Its job in this project: This is the thing being tested, packaged and deployed. Every change to it starts the pipeline.

The minute details

- Organised into folders such as **app/** (the code itself) and **tests/** (the Pytest tests).
- Dependencies (other libraries the code needs) are listed in a file like **requirements.txt** or **pyproject.toml**, so every machine installs exactly the same versions.
- Configuration (database address, feature switches) is read from **environment variables**, not written into the code. This lets the same code run in testing, staging and production with different settings.
- The app exposes a **health endpoint** (for example /health) that simply answers "I'm OK". Kubernetes and monitoring use this to check the app is alive.

Why it matters: Keeping code, dependencies and settings cleanly separated is what makes it possible to test and deploy it automatically.

3.2. Git and GitHub (code home and collaboration)

In one line: Git tracks every change to the code; GitHub stores the code online and is where the team reviews and merges changes.

Everyday analogy: A shared recipe book with a full history of every edit, plus a notice board where chefs ask others to approve their changes.

Its job in this project: The starting point of the pipeline. Every push and pull request on GitHub triggers GitHub Actions.

The minute details

- **Feature branches:** each developer works on a separate branch (e.g. feature/add-login), so unfinished work never breaks main.

- **Pull Requests:** when the work is ready, the developer opens a PR. The PR page shows the changes, the automated check results and reviewers' comments in one place.
- **Branch protection rules** on main, which make the process enforceable rather than optional:
  - No one can push directly to main.
  - All required checks (lint, tests, coverage) must pass (a green tick).
  - At least one reviewer must approve.
  - The branch must be up to date with main before merging.
- **Small, frequent PRs:** a key habit for daily shipping. Small changes are faster to review, faster to test and easier to undo.
- **Commit history** gives a clear record of who changed what and why, which helps when tracking down a bug.

Why it matters: GitHub turns good practice into rules. Nothing reaches main, and therefore users, without passing tests and a human review.

3.3. Pytest (the testing framework)

In one line: A Python tool for writing automatic tests and running them with a single command: pytest.

Everyday analogy: A set of taste tests written down so anyone (or any robot) can repeat them exactly, every time.

Its job in this project: Provides all the automated checks that prove the code works. Its integration tests were central to raising confidence enough to ship daily.

The minute details

- **Test files** live in tests/ and are named test_*.py. Each test is a small function whose name starts with test_.
- **Assertions:** a test uses assert to state what should be true, e.g. assert total == 100. If it isn't true, the test fails and shows exactly why.
- **Unit tests** check one function on its own. They run in milliseconds, so there can be hundreds of them.
- **Integration tests** check real pieces working together: the API talking to a real database, or one service calling another. These catch bugs unit tests miss, such as a wrong database query.
- **Fixtures** are reusable setup helpers. For example, a fixture can start a fresh test database, fill it with sample data, hand it to the test, and clean it up afterwards. Shared fixtures go in **conftest.py**.
- **Markers** label tests, e.g. @pytest.mark.integration, so the pipeline can run fast unit tests first and slower integration tests next.
- **Parametrize** runs the same test with many inputs, covering many cases with little code.

- **Mocks** stand in for outside services that cannot be called during tests (like a payment provider), so tests stay fast and reliable.
- **pytest-xdist** (optional) runs tests in parallel across CPU cores to keep the pipeline fast.

Example: a simple Pytest integration test using a database fixture

```
# tests/conftest.py
import pytest

@pytest.fixture
def db():
    conn = connect_to_test_database()   # real Postgres in Docker
    create_tables(conn)
    yield conn                          # hand it to the test
    drop_tables(conn)                   # clean up afterwards

# tests/test_orders.py
@pytest.mark.integration
def test_order_is_saved_and_read_back(db, api_client):
    resp = api_client.post("/orders", json={"item": "book", "qty": 2})
    assert resp.status_code == 201
    row = db.execute("SELECT qty FROM orders").fetchone()
    assert row.qty == 2
```

Why it matters: Tests are what make speed safe. Without trustworthy automated tests, shipping daily would simply mean shipping bugs daily.

3.4. Test Coverage and the 85% Gate (pytest-cov)

In one line: A measurement of how much of the code the tests actually run, plus a rule that the number must stay at 85% or higher.

Everyday analogy: If a recipe has 100 steps and your taste tests check 85 of them, your coverage is 85%. The gate is a rule that no dish leaves the kitchen if fewer than 85 steps were checked.

Its job in this project: Proves the testing is thorough and stops coverage from quietly slipping as new code is added.

The minute details

- **pytest-cov** watches which lines of code run while the tests run, then reports a percentage per file and overall.
- Run with: pytest --cov=app --cov-report=xml --cov-fail-under=85. The last option makes the command fail if coverage is under 85%.
- **How 85% was reached:**
  - Measured the starting coverage to find untested areas.
  - Prioritised the riskiest, most-used code first (payments, data saving, login), not easy code that pads the number.

  - Added integration tests for real user journeys end to end.
  - Set a rule that every new feature or bug fix must come with tests.
  - Raised the gate step by step (a “ratchet”) so the number could only go up, never down, until it reached 85%.
- **Reports** are uploaded as pipeline artifacts (and optionally to a service like Codecov) so reviewers can see which new lines are untested right on the PR.
- **Important honesty point:** coverage shows code was run, not that it was checked well. That is why code reviews also look at the quality of the tests, not just the number.

Why it matters: The gate turns “we should write tests” into an automatic rule, which is how coverage was raised and then kept at 85%.

3.5. Linting and Static Checks

In one line: Tools that read the code without running it, to catch style problems and simple mistakes.

Everyday analogy: A spell-checker and grammar-checker for code.

Its job in this project: The first and fastest stage of the pipeline. It fails in seconds if something obvious is wrong, before slower tests run.

The minute details

- **Formatter** (e.g. black): makes all code look the same, so reviews focus on logic, not spacing.
- **Linter** (e.g. flake8 or ruff): finds unused variables, undefined names, overly complex functions.
- **Type checker** (e.g. mypy): checks values are used as the right type, catching a whole class of bugs early.
- **Pre-commit hooks** can run the same checks on the developer’s laptop before code is even pushed.

Why it matters: Cheap, instant checks remove noise from code review and save pipeline time.

3.6. Docker (containers)

In one line: Packages the app together with Python, its libraries and settings into one sealed, portable box called an image.

Everyday analogy: A sealed lunchbox. Whatever is inside tastes the same whether you open it at home, at work or at a friend’s house.

Its job in this project: Used twice: (1) during testing, to start a real database for integration tests; (2) for release, to build the image that Kubernetes runs.

The minute details

- **Dockerfile:** a step-by-step recipe: start from a Python base image, copy requirements, install them, copy the code, define the start command.
- **Image:** the result of building the Dockerfile. It is read-only and identical everywhere.
- **Container:** a running copy of an image. You can run many containers from one image.
- **Layers and caching:** each Dockerfile line creates a layer. Unchanged layers are reused, so rebuilding after a small code change takes seconds.
- **Multi-stage builds:** build tools are used in one stage and left out of the final image, keeping it small and safer.
- **Tags:** each image is labelled with a version, usually the Git commit ID (e.g. app:3f9c2a1). This links every running app back to the exact code that produced it.
- **Docker Compose / service containers:** start helper containers (such as PostgreSQL or Redis) next to the tests so integration tests use real services.
- **Fixes "works on my machine":** the same image is tested in CI, tried in staging and run in production.

Example: a small multi-stage Dockerfile

```
FROM python:3.12-slim AS build
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.12-slim
WORKDIR /app
COPY --from=build /install /usr/local
COPY app/ ./app
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Why it matters: Docker makes the thing that was tested exactly the thing that is released, which removes a major source of release-day surprises.

3.7. GitHub Actions (the CI/CD engine)

In one line: GitHub's built-in automation service. It runs a list of steps automatically whenever something happens in the repository.

Everyday analogy: A robot kitchen inspector who appears the moment a dish is ready, runs every check in the same order, and posts a pass or fail note on the order board.

Its job in this project: Runs the whole pipeline: lint → unit tests → integration tests → coverage gate → build image → push to registry → deploy to Kubernetes.

The minute details

- **Workflow file:** a YAML file stored in .github/workflows/ (e.g. ci.yml). Because it lives with the code, the pipeline itself is versioned and reviewed like code.
- **Triggers (on:):** pull_request runs the checks on every PR; push to main runs the checks plus build and deploy.
- **Jobs:** large groups of work (e.g. test, build, deploy). Jobs can run in parallel, or wait for each other using needs:.
- **Steps:** the individual commands inside a job, such as checking out code, installing Python and running pytest.
- **Runners:** fresh, clean virtual machines provided by GitHub (e.g. ubuntu-latest). A new one is used every time, so results never depend on leftovers from earlier runs.
- **Actions (reusable steps):** ready-made building blocks, e.g. actions/checkout, actions/setup-python, docker/build-push-action.
- **Service containers:** start a PostgreSQL container next to the job so integration tests hit a real database.
- **Caching:** pip packages and Docker layers are cached between runs to cut minutes off each pipeline.
- **Matrix builds** (optional): run the tests on several Python versions at once.
- **Secrets:** registry passwords and the Kubernetes access key are stored encrypted in GitHub Secrets and never appear in logs.
- **Environments:** staging and production can be set up as GitHub Environments, with optional manual approval before production.
- **Status checks:** results appear on the PR as green ticks or red crosses and are required by branch protection.

Example: a simplified .github/workflows/ci-cd.yml

```
name: ci-cd
on:
  pull_request:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_PASSWORD: test }
        ports: ["5432:5432"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12", cache: pip }
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: ruff check . && black --check .
      - run: pytest -m "not integration"
      - run: pytest -m integration --cov=app --cov-fail-under=85

  build-and-deploy:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - uses: docker/build-push-action@v6
        with:
          push: true
          tags: ghcr.io/org/app:${{ github.sha }}
      - name: Deploy to staging
        run: |
          IMG=ghcr.io/org/app:${{ github.sha }}
          kubectl -n staging set image deploy/app app=$IMG
          kubectl -n staging rollout status deploy/app
          ./scripts/smoke_test.sh staging
      - name: Promote to production
        run: |
          IMG=ghcr.io/org/app:${{ github.sha }}
          kubectl -n production set image deploy/app app=$IMG
          kubectl -n production rollout status deploy/app
```

Why it matters: Automation is the heart of the speed-up. The same checks run on every change, within minutes, without anyone remembering to run them.

3.8. Container Registry

In one line: An online warehouse that stores every Docker image the pipeline builds, labelled by version.

Everyday analogy: A labelled pantry. Every lunchbox ever packed sits on a shelf with a date and recipe number, ready to be served or served again.

Its job in this project: The hand-off point between CI and CD. GitHub Actions puts images in; Kubernetes takes images out.

The minute details

- Common choices: GitHub Container Registry (ghcr.io), Docker Hub, AWS ECR, Google Artifact Registry.
- Each image is tagged with the commit ID, so you always know exactly what code is in it.
- Old images are kept, which makes **rollback** simple: tell Kubernetes to run the previous tag.
- Images can be **scanned** for known security issues before being deployed.
- Access is protected; the pipeline logs in using a stored secret.

Why it matters: Keeping every version ready and labelled makes deployments repeatable and rollbacks instant.

3.9. Kubernetes (running the app in production)

In one line: A system that runs containers across a group of machines and keeps the right number of healthy copies running at all times.

Everyday analogy: A restaurant manager. They decide how many waiters to have on shift, replace anyone who falls ill, add staff when it gets busy, and swap in the new menu table by table without closing the doors.

Its job in this project: Takes the tested Docker image and runs it for real users with zero downtime, first in staging and then in production.

The minute details

- **Cluster:** the group of machines (called **nodes**) that Kubernetes manages.
- **Pod:** the smallest unit Kubernetes runs; here, one container of the app.
- **Deployment:** a file that says, for example, “always keep 3 pods of image app:3f9c2a1 running”. Kubernetes constantly makes reality match that wish.
- **ReplicaSet:** the helper the Deployment uses to keep the exact number of pods alive. If a pod crashes, a new one starts automatically.
- **Service:** a stable address that spreads traffic across all healthy pods, so users never need to know which pod they reach.

- **Ingress:** the front door that routes outside web traffic to the right Service.
- **Namespaces:** separate areas in one cluster. Here: **staging** (practice) and **production** (live).
- **ConfigMaps and Secrets:** hold settings and passwords outside the image, so one image works in every environment.
- **Readiness probe:** Kubernetes asks /health "are you ready?" and only sends traffic to pods that answer yes.
- **Liveness probe:** Kubernetes regularly asks "are you still alive?" and restarts pods that stop answering.
- **Rolling update:** new pods start, pass their readiness check, then old pods are removed a few at a time. Users never see an outage.
- **Rollback:** kubectl rollout undo returns to the previous version in seconds if something goes wrong.
- **Autoscaling (HPA):** adds pods when traffic is high and removes them when it is quiet.

Example: a simplified Kubernetes Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: app, namespace: production }
spec:
  replicas: 3
  strategy:
    rollingUpdate: { maxUnavailable: 0, maxSurge: 1 }
  selector: { matchLabels: { app: app } }
  template:
    metadata: { labels: { app: app } }
    spec:
      containers:
        - name: app
          image: ghcr.io/org/app:3f9c2a1
          ports: [{ containerPort: 8000 }]
          readinessProbe: { httpGet: { path: /health, port: 8000 } }
          livenessProbe:  { httpGet: { path: /health, port: 8000 } }
```

Why it matters: Zero-downtime rollouts and one-command rollback make each release low-risk, which is what allows releasing every day.

3.10. Monitoring and Alerting

In one line: Tools that watch the live app and raise an alarm if something goes wrong after a release.

Everyday analogy: A smoke detector and a customer feedback card in the restaurant.

Its job in this project: The final safety net. It confirms each release is healthy and triggers a rollback if it is not.

The minute details

- **Health checks:** Kubernetes probes plus external checks that hit the app's /health page.

- **Metrics** (e.g. Prometheus and Grafana): error rate, response time, CPU and memory, shown on dashboards.
- **Logs:** every pod's output is collected in one searchable place.
- **Alerts:** if errors spike after a release, the team gets a message (e.g. Slack or PagerDuty).
- **Rollback decision:** an unhealthy release is undone with a single command, or automatically if probes fail during the rollout.

Why it matters: Fast detection plus fast rollback means a bad release causes minutes of trouble, not days.

3.11. Design Reviews and Code Reviews (the human side I led)

In one line: Structured conversations where the team checks the plan before building (design review) and checks the code before merging (code review).

Everyday analogy: The head chef approves the new menu idea before cooking starts, and a second chef checks every plate before it goes out.

Its job in this project: Automation catches mechanical mistakes; reviews catch wrong ideas, unclear code, missing tests and risky designs. I led both.

The minute details

- **Design review (before coding):**
  - The author writes a short design doc: the problem, the proposed solution, alternatives, risks, how it will be tested and how it will be rolled out.
  - The team discusses it and agrees before effort is spent writing code.
  - In this project, design reviews decided things like the pipeline stages, the testing strategy, the 85% target and the staging-then-production rollout.
- **Code review (on every PR):**
  - At least one teammate reads the change and approves or requests fixes.
  - A shared checklist: Is it correct? Is it readable? Does it have meaningful tests? Is it secure? Does it handle errors?
  - Reviews happen quickly (same day) so they do not slow down daily shipping.
  - Small PRs are encouraged so each review is short and focused.
- **Leading** meant setting the standards, writing the checklists and PR templates, mentoring others on writing tests, and keeping review turnaround fast.

Why it matters: Reviews spread knowledge across the team and catch the problems tools cannot, keeping quality high while speed increases.

4. The End-to-End Workflow, Step by Step

This section follows one code change from the developer's keyboard all the way to real users. The step numbers match the numbers in the architecture diagram in Section 5.

> **The whole journey in one line**
> Write code + tests → check locally → push to a branch → open a PR → robot runs lint, unit tests, integration tests and the 85% coverage check → teammate reviews → merge to main → build Docker image → store it in the registry → deploy to staging and smoke-test → roll out to production → users get the update → monitoring watches and rolls back if needed.

4.1 Phase A — Writing the change (Zone 1)

Step 1 — Developer writes code and tests

What happens: The developer creates a feature branch and writes the new code together with its Pytest tests.

In simple words: The chef cooks a new dish and writes down the taste tests for it at the same time.

- Tests are written alongside the code, not later, so coverage never falls behind.
- For larger changes, the design has already been agreed in a design review.

Step 2 — Local checks

What happens: The developer runs pytest, the linter and optionally docker build on their own laptop.

In simple words: The chef tastes the dish before handing it over.

- Pre-commit hooks can run formatting and linting automatically on every commit.
- Catching problems here is the cheapest place to fix them.

If it goes wrong: The developer fixes the problem on the spot; nothing has left their laptop yet.

4.2 Phase B — Sharing the change (Zone 2)

Step 3 — Push to a feature branch on GitHub

What happens: git push sends the commits to the developer's branch on GitHub.

In simple words: The dish is placed on the pass, in its own spot, without touching anyone else's.

- main is untouched, so other developers are not affected by unfinished work.

Step 4 — Open a Pull Request

What happens: The developer opens a PR asking to merge the branch into main, using a PR template (what changed, why, how it was tested).

In simple words: The chef rings the bell: “this dish is ready for inspection”.

- Opening the PR automatically **triggers GitHub Actions** (the pull_request trigger).
- Reviewers are assigned automatically or by the author.

4.3 Phase C — Automatic checks (Zone 3, the CI pipeline)

GitHub Actions starts a fresh runner machine, downloads the code, sets up Python, restores the cache and installs dependencies. Then it runs the stages below in order. Each stage only runs if the previous one passed, so failures are reported as early as possible.

Step 5 — Lint and style check

What happens: black, ruff/flake8 and mypy read the code for style issues and simple bugs.

In simple words: A spell-check for code. It takes seconds.

- Failing fast here saves the minutes the test stages would otherwise use.

If it goes wrong: Red cross on the PR with the exact file and line to fix.

Step 6 — Unit tests

What happens: pytest -m "not integration" runs all the fast, isolated unit tests.

In simple words: Taste each ingredient on its own.

- Hundreds of tests finish in well under a minute; they can run in parallel with pytest-xdist.

If it goes wrong: Pytest shows which test failed, what it expected and what it got.

Step 7 — Integration tests

What happens: GitHub Actions starts a real PostgreSQL (and any other needed service) as a Docker service container. pytest -m integration then runs tests that use the real database and real API calls.

In simple words: Taste the whole plate to make sure the ingredients work together.

- Fixtures create a clean database before each test and clean it up after, so tests never affect each other.
- These tests catch problems unit tests cannot, such as broken database queries or mismatched data formats.

If it goes wrong: The PR is blocked and the logs show the failing request or query.

Step 8 — Coverage gate (85%)

What happens: pytest-cov measures coverage across all tests. --cov-fail-under=85 fails the run if coverage is below 85%.

In simple words: Confirm that at least 85 out of every 100 lines were actually tasted.

- The coverage report is saved and shown on the PR, highlighting new lines with no tests.
- When all stages pass, GitHub shows a **green tick** on the PR.

Step 9 — Failure path: feedback to the developer

What happens: If any of steps 5–8 fails, the pipeline stops, the PR shows a red cross, and the developer is notified.

In simple words: The inspector sends the dish back with a note saying exactly what is wrong.

- Branch protection makes it impossible to merge while checks are red.
- The developer fixes the issue, pushes again, and the pipeline re-runs automatically from step 5.

4.4 Phase D — Human review and merge (back in Zone 2)

Step 10 — Code review

What happens: A teammate reviews the PR using the team checklist, leaves comments, and approves when satisfied.

In simple words: A second chef checks the plate looks right and makes sense.

- Reviewers check logic, readability, security, error handling and whether the tests are meaningful.
- Fast turnaround (same day) keeps the daily release rhythm going.

If it goes wrong: The reviewer requests changes; the author updates the PR and the automated checks re-run.

Step 11 — Merge into main

What happens: With green checks and an approval, the PR is merged. This push to main triggers the delivery part of the pipeline.

In simple words: The dish is approved and officially added to the menu.

- main stays “always releasable” because only fully checked and reviewed code gets in.

4.5 Phase E — Packaging and delivery (Zones 3, 4 and 5)

Step 12 — Build the Docker image

What happens: GitHub Actions builds the image from the Dockerfile and tags it with the commit ID (e.g. app:3f9c2a1).

In simple words: Seal the approved dish in a labelled lunchbox.

- Cached layers make the build fast.
- The image can be scanned for known security vulnerabilities at this point.

Step 13 — Push the image to the container registry

What happens: The pipeline logs in using a stored secret and uploads the image.

In simple words: Put the labelled lunchbox on the pantry shelf.

- Previous images stay on the shelf, ready for an instant rollback.

Step 14 — Deploy to staging and run smoke tests

What happens: The pipeline tells Kubernetes (staging namespace) to run the new image, waits for the rollout to finish, then runs quick smoke tests against it.

In simple words: Serve the dish at a practice table first and make sure it is fine.

- kubectl set image (or Helm / a GitOps tool) updates the Deployment; kubectl rollout status waits until all new pods are ready.
- Smoke tests hit key pages and endpoints (health, login, a main feature).

If it goes wrong: The pipeline stops, production is never touched, and the team is alerted.

Step 15 — Promote to production with a rolling update

What happens: The same image is deployed to the production namespace. Kubernetes starts new pods, waits until they pass their readiness checks, then removes old pods one at a time.

In simple words: Swap in the new dish table by table without ever closing the restaurant.

- Optionally, a GitHub Environment can require a one-click manual approval before this step.
- With maxUnavailable set to 0, capacity never drops during the rollout.

If it goes wrong: If new pods fail their health checks, Kubernetes stops the rollout and old pods keep serving users.

4.6 Phase F — Users and safety net (Zone 6)

Step 16 — End users get the new version

What happens: Traffic flows through the Ingress and Service to the new healthy pods.

In simple words: Customers are now eating the new dish, and nobody noticed the switch.

- This whole journey, from merge to users, now takes minutes and happens every day.

Step 17 — Monitoring, alerts and rollback

What happens: Monitoring watches error rates, response times and logs. If something looks wrong, an alert goes to the team, and the release is rolled back.

In simple words: The smoke detector is always on; if it beeps, we instantly bring back yesterday's dish.

- Rollback: kubectl rollout undo deploy/app, or redeploy the previous image tag.
- The fix then goes through the same pipeline from step 1, so even emergency fixes are tested.

4.7 A typical day, before and after

Time	Before (weekly)	After (daily)
Morning	Developers keep adding changes to a big pile for Friday	A developer opens a small PR; checks finish in minutes
Midday	No one knows yet if the pile works together	A teammate reviews and approves; PR merged
Afternoon	Manual testing of the whole week's work begins	Image built, staged, smoke-tested and rolled out automatically
Release	Friday: long, risky release; bugs found late	Several small, safe releases happen through the day
If a bug appears	Hard to find among a week of changes	Easy to find in one small change; rollback in seconds

5. Architecture Diagram

The diagram shows every part of the system and the path a code change takes. Coloured boxes are zones (areas of responsibility). Numbers in brackets match the workflow steps in Section 4. Solid arrows are the normal flow; the green arrow is the “all checks passed” path; the red dashed arrow is the failure path.

5.1 How to read the diagram

Visual element	Meaning
Large coloured rounded box	A zone: one area of responsibility (laptop, GitHub, CI, registry, Kubernetes, monitoring)
White rounded box	One component or step inside a zone
Number in brackets, e.g. (7)	The step number; matches Section 4
Solid grey arrow	Normal flow: the code or image moves to the next step
Solid green arrow	Checks passed: the PR is cleared for human review
Red dashed arrow	Something failed: the change goes back to the developer
Orange-filled box (8)	The coverage gate, the key quality checkpoint
Oval	People outside the system (end users)
Folded-corner note (9)	An outcome rather than a system: feedback to the developer

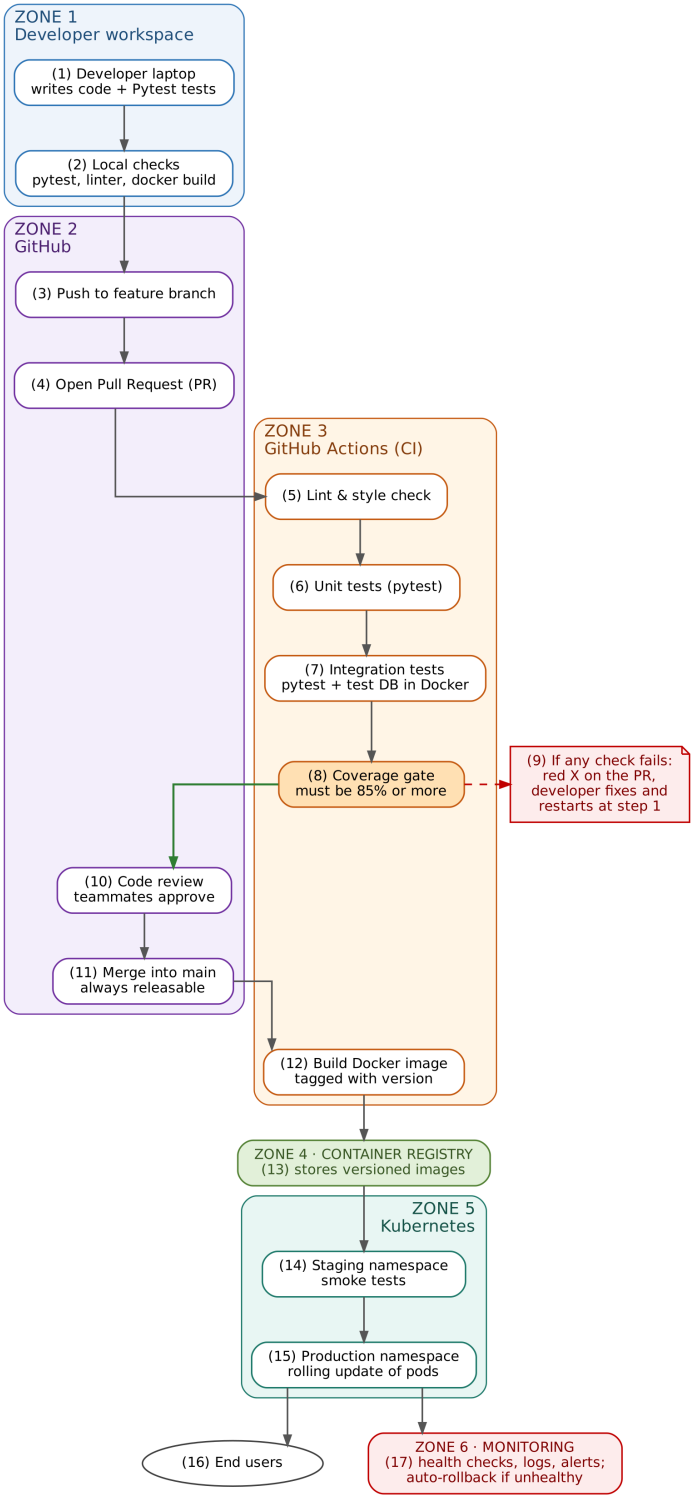


Figure 1. End-to-end CI/CD architecture

5.2 Zone-by-zone explanation

Zone 1 — Developer workspace (blue)

Contains steps (1) and (2). This is the developer's own computer, where code and Pytest tests are written and checked locally with pytest, the linter and docker build. Nothing here affects anyone else. It is the cheapest and fastest place to find a mistake.

Zone 2 — GitHub (purple)

Contains steps (3), (4), (10) and (11). GitHub stores the code. A change arrives on a feature branch (3), becomes a pull request (4), returns here after automated checks for human code review (10), and is finally merged into main (11). Branch protection rules on main sit over this whole zone and guarantee no change skips the checks or the review. Notice that this zone appears twice in the flow: once before the robot checks and once after.

Zone 3 — GitHub Actions CI pipeline (orange)

Contains steps (5) to (8) and (12). This is the automation engine running on fresh GitHub runner machines. The PR trigger runs the quality stages in order: lint (5), unit tests (6), integration tests with a real database in Docker (7) and the 85% coverage gate (8). After the merge, the same zone builds the Docker image (12). The zone is tall because it serves both halves of the journey: checking a PR and packaging a merged change.

Failure note (red, step 9)

Sits outside all zones on purpose: it is not a system but a result. If any stage from (5) to (8) fails, the red dashed arrow leads here: the PR gets a red cross, merging is blocked, and the developer fixes the issue and pushes again, restarting at step (1).

Zone 4 — Container registry (green)

Step (13). A single box because it has one job: storing every built image, tagged with its commit ID. It is the hand-over point between CI (building) and CD (deploying). Old images stay here so rolling back is instant.

Zone 5 — Kubernetes cluster (teal)

Steps (14) and (15). Two namespaces in the same cluster. Staging (14) receives the image first and is checked with smoke tests. Only if that passes is the exact same image promoted to production (15), where Kubernetes performs a rolling update so users never see downtime.

End users (oval, step 16)

Real people using the app. They are reached only from production, never from staging or CI.

Zone 6 — Monitoring (red, step 17)

Watches production health: probes, metrics, logs and alerts. It is coloured red like the failure note because both are safety nets. If production becomes unhealthy, monitoring alerts the team and triggers a rollback to the previous image from the registry.

5.3 Every arrow explained

Arrow	From → To	What travels along it	Plain meaning
1 → 2	Laptop → Local checks	Source code + tests	Developer checks their own work first
2 → 3	Local checks → Feature branch	Git commits (git push)	Work is shared on GitHub, separate from main
3 → 4	Branch → Pull Request	A request to merge	Developer asks for the change to be checked
4 → 5	Pull Request → Lint	A trigger event (pull_request)	Opening the PR wakes up the robot
5 → 6	Lint → Unit tests	Passed status	Style is fine; now test small pieces
6 → 7	Unit tests → Integration tests	Passed status	Small pieces work; now test them together
7 → 8	Integration tests → Coverage gate	Coverage data	Check enough of the code was tested
8 → 10 (green)	Coverage gate → Code review	Green tick on the PR	Robot is happy; a human now reviews
8 → 9 (red dashed)	Any check → Feedback note	Red cross + error log	Something failed; developer must fix it
10 → 11	Code review → main	Approval, then merge	Reviewed change becomes official
11 → 12	main → Build image	A trigger event (push to main)	Merging starts the packaging
12 → 13	Build → Registry	Tagged Docker image	The packaged app is stored by version

Arrow	From → To	What travels along it	Plain meaning
13 → 14	Registry → Staging	Image pulled by Kubernetes	Try the new version in a safe copy
14 → 15	Staging → Production	Same image, promoted	Smoke tests passed; release to real users
15 → 16	Production → End users	Live web traffic	Users get the new version, no downtime
15 → 17	Production → Monitoring	Health data, metrics, logs	System is watched; rollback if unhealthy

5.4 Why the architecture is shaped this way

- **Fail early, fail cheap:** fast checks (lint, unit tests) come before slow ones (integration tests, deploy), so most problems are caught in seconds.
- **Machines first, humans second:** reviewers only spend time on PRs that already pass every automated check.
- **Build once, deploy many:** one image is built and the exact same image goes to staging and production, so what was tested is what runs.
- **Staging before production:** a practice run catches environment problems before users see them.
- **Separation of CI and CD by the registry:** building and deploying are decoupled, which makes rollback trivial and audits easy.
- **A safety net at both ends:** the failure path protects main; monitoring and rollback protect users.

6. How the Two Results Were Achieved

6.1 Raising test coverage to 85%

1. Measured the starting point with pytest-cov to see which files and functions had no tests.
2. Agreed the target and the approach in a design review, so the whole team was on board.
3. Wrote integration tests for the most important user journeys first (where bugs would hurt most).
4. Built shared Pytest fixtures (test database, API client, sample data) so writing new tests became quick and easy.
5. Added the coverage gate to GitHub Actions and raised it gradually, so it could only go up.
6. Made “tests included” a standard item in the code review checklist.

6.2 Moving from weekly to daily releases

What made weekly releases slow	What fixed it
Manual testing took days	Automated Pytest suite runs in minutes on every change
Large batches of changes were risky	Small PRs, each tested and released on its own
Differences between laptops, test and production	One Docker image used everywhere
Deploying caused downtime and stress	Kubernetes rolling updates with health checks
Fixing a bad release was slow	Versioned images and one-command rollback
Reviews were inconsistent or slow	Clear review checklists, PR templates and same-day turnaround

7. Explaining It in an Interview

7.1 The 30-second version

> **Say it like this**
>
> "Our team was shipping weekly with a lot of manual testing. I set up a CI/CD pipeline in GitHub Actions: every pull request runs linting, unit tests and Pytest integration tests against a real database in Docker, and fails if coverage drops below 85%. Once a change is reviewed and merged, the pipeline builds a Docker image, pushes it to our registry, deploys to a staging namespace in Kubernetes, runs smoke tests, then rolls out to production with zero downtime. I also led design and code reviews to keep quality high. Coverage went to 85% and we moved from weekly to daily releases."

7.2 Likely follow-up questions

Question	Short answer
Why integration tests and not just unit tests?	Unit tests prove pieces work alone; integration tests prove they work together with a real database, which is where many real bugs hide.
Is 85% coverage enough? Why not 100%?	85% covers the important code with reasonable effort. The last 15% is often trivial or very hard to test, and chasing 100% gives poor returns. Reviews check test quality, not just the number.
How did you keep the pipeline fast?	Ran fast checks first, cached dependencies and Docker layers, ran tests in parallel and separated unit from integration tests with markers.
How do you handle flaky tests?	Isolate data with fixtures, avoid timing-based waits, quarantine and fix flaky tests quickly rather than ignoring them.
How do you roll back?	Every image is tagged with its commit ID; kubectl rollout undo or redeploying the previous tag restores the last good version in seconds.
How are secrets handled?	Stored in GitHub Secrets and Kubernetes Secrets, never in code or logs.
What did leading reviews involve?	Writing design-doc and PR templates, setting a review checklist, mentoring on testing and keeping review turnaround to the same day.